L09. Arithmetic Logic Design (1)

Lecture 09

9.1 설계 절차 (교재 5.1, 5.5)

> **설계 = 사양을 게이트 수준 구현으로 사상하는 일.**

Design = 회로 specification 을 게이트 수준 구현으로 mapping 하는 것.

단계	내용	세부
Step 1	문제를 이해한다	입력/출력 사양
Step 2	표준 표현으로 정식화한다	부울식 또는 진리표
Step 3	구현 방식을 고른다	이산 게이트의 2단 논리 / PLA, ROM, FPGA …
Step 4	설계 절차를 적용한다	2단 논리: K-map 으로 최적화. ROM 기반 설계: 진리표만 있으면 됨. 다단 논리: CAD 소프트웨어

설계 예: Logic Function Unit

사양: 데이터 입력 A, B, 제어 입력 C_0, C_1, C_2 셋. 논리 함수 F 는 제어 입력에 따라 아래 표처럼 정해짐.

C_0	C_1	C_2	F	설명
0	0	0	1	Always 1
0	0	1	A + B	Logical OR
0	1	0	(AB)′	Logical NAND
0	1	1	A ⊕ B	Logical XOR

C_0	C_1	C_2	F	설명
1	0	0	A ⊙ B (XNOR)	Logical XNOR
1	0	1	AB	Logical AND
1	1	0	(A + B)′	Logical NOR
1	1	1	0	Always 0

Step 1~2 (문제 이해, 정식화)	위의 표. 입력 5개(C_0, C_1, C_2, A, B): 제어 조합 8가지 × 데이터(A, B) 조합 4가지 = 32행 진리표.
Step 3 (구현 방식)	8:1 MUX 한 개를 함수 선택부로 사용한다. 선택 신호는 C_0, C_1, C_2이고, 데이터 입력 D_0~D_7에는 각각 1, A+B, (AB)′, A⊕B, A⊙B, AB, (A+B)′, 0을 연결한다. 따라서 MUX 외에도 OR, NAND, XOR, XNOR 등 필요한 데이터 입력을 생성하는 조합논리 회로가 필요하다.
Step 4 (구현 절차)	입력이 C_0, C_1, C_2, A, B인 5변수 함수이므로 총 32셀의 K-map을 사용한다. C_0=0과 C_0=1에 해당하는 4변수 K-map 두 개를 만들고, 각 맵의 행에는 C_2C_1, 열에는 AB를 Gray code 순서 00, 01, 11, 10으로 배치한다. 두 맵에서 인접한 항까지 묶어 최소화하면 $F = C_2'A'B' + C_0'(A⊕B) + C_1'AB$를 얻는다.

결과

$$F = C_2'A'B' + C_0'AB' + C_0'A'B + C_1'AB$$
$$= C_2'A'B' + C_0'(A \oplus B) + C_1'AB$$

9.2 가산기 설계 (교재 5.6)

Half adder 와 Full adder

Half adder	$S = A \oplus B$, $C_{out} = A \cdot B$.
Full adder	세 번째 입력 C 는 이전 자리의 carry-in(C_{in}). $S = A \oplus B \oplus C$, $C_{out} = MAJ(A,B,C) = AB + BC + CA$. 입력 세 개 중 1이 홀수 개면 S = 1, 2개 이상이면

$C_{out} = 1$.

A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

A	B	C	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

다비트 가산기

- 4-bit 가산기는 두 4비트 피연산자 A[3:0], B[3:0]와 1비트 carry-in C_{in}을 더한다. 전체 연산 결과는 $\{C_{out}, S[3:0]\} = A[3:0] + B[3:0] + C_{in}$으로 표현한다. 하위 4비트가 합 S[3:0]이고, 다섯 번째 비트가 carry-out C_{out}이다.
- full adder 블록 4개로 구성하고, 앞 블록의 carry-out 이 다음 블록의 carry-in.

예 1 · +4 + +3

A = +4 = 0100_2, B = +3 = 0011_2, CI = 0. 자리마다 carry 0, Sum = 0111_2 = +7.

$C_{O3} = 0$, $C_{O4} = 0$ (C_{O3}: MSB 로 들어가는 carry, C_{O4}: MSB 에서 나오는 carry). Overflow = $C_{O3} \oplus C_{O4} = 0$.

예 2 · +7 + +1 (overflow)

A = +7 = 0111_2, B = +1 = 0001_2, CI = 0. 아래 자리부터 carry 가 전부 올라가서 Sum = 1000_2.

4비트 2의 보수에는 +8 이 없음. 1000_2 = −8 ≠ +8. C_{O3} = 1, C_{O4} = 0, Overflow = $C_{O3} \oplus C_{O4}$ = 1.

- n비트 2의 보수(n = 비트 수): $N^* = (-)N = 2^n - N = \bar{N} + 1$ ($\bar{N}$는 비트 반전). 결과는 n비트만 남김(mod 2^n). N = 0이면 결과도 0.
- 4비트 표현: −8 = 1000, −7 = 1001, −6 = 1010, −5 = 1011, −4 = 1100, −3 = 1101, −2 = 1110, −1 = 1111, 0 = 0000, 1 = 0001 ⋯ 7 = 0111.

2의 보수 변환기

체계	해석값 (1101)	표현 범위 (4비트)	0의 개수
Unsigned	13	0 ~ 15	1개
Sign-magnitude	−5	−7 ~ 7	2개 (+0, −0)
1의 보수	−2	−7 ~ 7	2개
2의 보수	−3	−8 ~ 7	1개

2의 보수 부호 반전은 n비트에서 $-x \equiv$ 반전(x) + 1 (mod 2^n)로 계산한다. 예를 들어 4비트의 1101(−3)은 0010으로 비트 반전한 뒤 1을 더해 0011(+3)이 된다. 단, n비트 2의 보수의 최소 음수 $-2^{(n-1)}$는 반대값 $+2^{(n-1)}$을 같은 비트 수로 표현할 수 없으므로 예외이다. 예를 들어 4비트에서 1000(−8)을 반전한 뒤 1을 더하면 다시 1000이 되며, 이 경우 signed negation overflow가 발생한다.

부호 확장 (4→8비트): 부호 비트 1을 앞에 4개 복제 → 11111101 (값 −3 유지)

2의 보수 덧셈·뺄셈: carry 와 overflow

A + B → 실제 값 8, 비트 결과 1000 = −8

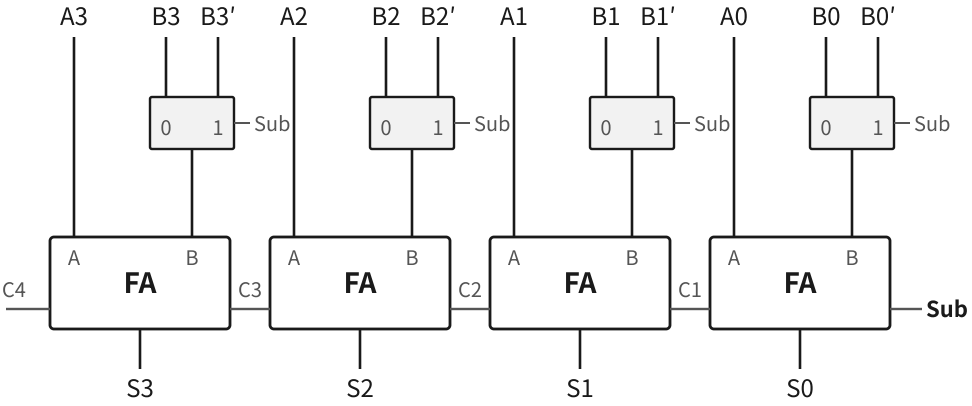
Cn−1 = 1, Cn = 0 → overflow = Cn−1 XOR Cn = 1

signed overflow: 실제 값 8이 4비트 범위 (-8 ~ 7) 밖이므로 비트 결과 −8가 실제 값과 다르다.

unsigned로 보면 7 + 1 = 8 (CO=0 → unsigned 범위 안). signed overflow와 unsigned overflow는 별도로 판정한다.

- Overflow 판정: C_3(MSB 로 들어가는 carry)와 C_4(MSB 에서 나오는 carry)가 다르면 overflow. 양수끼리 더했는데 부호 비트가 1, 또는 음수끼리 더했는데 0 이 되는 경우.

Adder / Subtractor



4-bit adder/subtractor. Sub = 0 이면 mux 가 B 를 고르고 C_{in} = 0 이라 A + B. Sub = 1 이면 B′ 를 고르고 C_{in} = 1 이라 A + B′ + 1 = A − B.

- Sub = 0 이면 덧셈, Sub = 1 이면 뺄셈. 비트마다 앞에 mux(Sel)가 B_i 와 B_i' 중 하나를 고르고, Sub 는 맨 아래 carry-in 에도 들어감.
- A − B = A + (−B) = A + ($\bar{B}$ + 1), 2의 보수 방식. B 를 반전하고 carry-in 을 1 로 두면 뺄셈. Overflow 는 마지막 두 carry 의 XOR.

9.3 Ripple carry adder 의 지연

- Ripple carry adder(RCA)는 carry 를 낮은 자리에서 높은 자리로 전파시킴. 이 전파가 critical delay이며 시스템 동작 속도를 결정.
- 32비트 RCA 는 최상위 carry 가 32개 full adder 의 carry 경로를 순차적으로 거쳐야 해서 느리다.

RCA 식

S = CI ⊕ A ⊕ B

CO = CI(A + B) + AB

(+ 와 곱은 논리 OR, AND)

- 단마다 carry 경로에 게이트 지연 2개가 더해짐(CO 에 누적). 4단 RCA 에서 C1 = @2, C2 = @4, C3 = @6, C4 = @8 이고 S0 는 @2 … S3 는 @7 이다 (@N 은 게이트 지연 N개 뒤).
- 16비트 RCA 는 지연이 @32 로 비효율적. 16번째 full adder 가 첫 번째 full adder 의 carry 를 기다려야 하기 때문. 앞 단을 안 기다리고 carry-out 을 미리 알아내는 방법이 필요.

9.4 Carry lookahead adder (CLA)

- 4-bit ripple adder에서는 C_{i+1}이 A_i, B_i, C_i에 의해 결정되므로 carry가 순차적으로 전파된다. 반면 carry lookahead adder는 C_{i+1}을 A_i, A_{i-1}, …, A_0, B_i, B_{i-1}, …, B_0, 초기 carry C_0의 함수로 직접 전개한다. 즉 C_i는 0~i−1번째 비트의 입력으로 결정되며, 앞 단계의 carry 출력을 순차적으로 기다리지 않고 계산할 수 있다.

	정의	의미
Carry Generate	$G_i = A_iB_i$	A = B = 1 이면 반드시 C_o 를 만든다
Carry Propagate	$P_i = A_i \oplus B_i$	A ≠ B 일 때 1. 이때 carry-in 이 그대로 carry-out 이 된다

A	B	CI	S	CO	P	G
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	1	0	1	0	1	0
0	1	1	0	1	1	0
1	0	0	1	0	1	0
1	0	1	0	1	1	0
1	1	0	0	1	0	1

A	B	CI	S	CO	P	G
1	1	1	1	1	0	1

> **Carry lookahead**
>
> $C_{i+1} = G_i + C_iP_i$ (+ 는 논리 OR, 곱은 논리 AND)
>
> $C_{i+1} = 1$ 인 경우: A_i 와 B_i 가 모두 1 (carry generate, G_i), 또는 C_i 가 전파될 때 (carry propagate, P_i).

C_i 를 반복 대입.

> $C_1 = G_0 + P_0C_0$
>
> $C_2 = G_1 + P_1C_1 = G_1 + P_1G_0 + P_1P_0C_0$
>
> $C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$
>
> $C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$

- carry 식은 P_i, G_i 를 쓴 2단 논리 네트워크로 구현(P_i, G_i 생성 논리는 따로 필요). 변수가 가산기 입력과 C_0 뿐이라 앞 단 carry-out 을 기다리지 않음.
- 지연. P_i, G_i 계산이 @1, lookahead 로 C_i 계산이 @2. 그래서 A_i, B_i, C_0 에서 C_i 까지 @1 + @2 = @3. $S_i = P_i \oplus C_i$ 는 C_i 뒤에 XOR @1 이 붙어 @4. (RCA 는 S3 가 @7.)
- $S_i = A_i \oplus B_i \oplus C_i = P_i \oplus C_i$.

Lookahead carry unit 구조 (4-bit CLA)

블록	식	지연
PG unit	$G_i = A_iB_i$, $P_i = A_i \oplus B_i$	@1
Lookahead Carry Unit	$C_1 = C_0P_0 + G_0$, $C_2 = C_0P_0P_1 + G_0P_1 + G_1$, ⋯	@2
SUM unit	$S_0 = C_0 \oplus P_0$, $S_1 = C_1 \oplus P_1$, ⋯	@1

Hierarchical CLA (16-bit)

- 그룹 propagate·generate 정의와 C_4 전개:

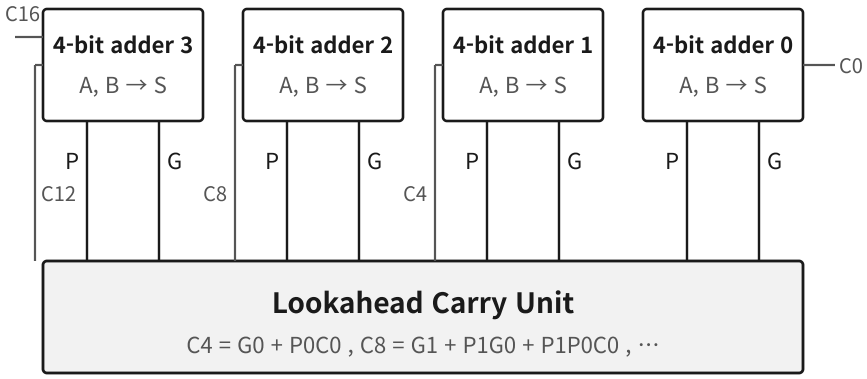
  그룹 propagate: $P_{3\text{-}0} = P_3P_2P_1P_0$

  그룹 generate: $G_{3\text{-}0} = G_3 + G_2P_3 + G_1P_3P_2 + G_0P_3P_2P_1$

  $C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$

  $C_4 = G_{3\text{-}0} + C_0 \cdot P_{3\text{-}0}$

- 16비트 가산기 = 4-bit 가산기 4개 + 4-bit Lookahead Carry Unit. 각 4-bit 가산기는 내부 carry lookahead 로 그룹 P, G 를 내보냄. 두 번째 레벨의 lookahead carry unit 이 이를 16비트로 확장 (C_4, C_8, C_{12}, C_{16}).



hierarchical CLA. 4-bit 블록이 P, G 를 내고 블록 사이의 carry(C4, C8, ⋯)는 Lookahead Carry Unit 이 한꺼번에 계산한다.

- 그룹 $P_{3\text{-}0}$ 은 AND 로 @1, 그룹 $G_{3\text{-}0}$ 은 OR-AND 로 @2. 첫 4-bit 블록의 $S_3 = P_3 \oplus C_3$ 는 C_3 = @3 에 XOR @1 을 더해 @4.

두 번째 블록	C_4(두 번째 블록의 carry-in)가 @4 에 나옴. 블록 안 lookahead 로 C_7 까지 @2, XOR @1 이 더해져 S_7 = @4 + @2 + @1 = @7.
네 번째 블록	carry-in C_{12} 는 @5, S_{15} = @5 + @2 + @1 = @8. RCA 16비트는 @32 였으니 차이가 큼.

9.5 Carry select adder

- carry 계산을 빠르게 하려고 하드웨어를 중복해서 씀.
- 상위 자리의 합을 병렬로 계산. 한쪽은 carry-in = 0, 다른 쪽은 carry-in = 1 로 가정. 실제 carry 가 도착하면 mux 로 맞는 쪽을 고름.
- hierarchical로 확장 가능.

9.6 BCD adder

- BCD(binary coded decimal): 10진수 0~9 를 4비트 이진수 0000~1001 로 표현. 예: 13 은 BCD 로 0001_0011.

> - 예: 5 + 8. 이진 합 1101(13)은 BCD 로는 틀림. 보정: 자릿수 합이 9 를 넘으면 6(0110_2)을 더함. 1101 + 0110 = 1_0011, 즉 0001_0011.

- BCD adder는 두 유효한 BCD 숫자(0~9)와 십진 carry-in을 일반 4비트 이진 가산기로 먼저 더한다. 첫 번째 가산기의 carry-out을 K_1, 하위 4비트 합을 $S_3S_2S_1S_0$라고 할 때, 보정 조건은 $K = K_1 + S_3S_2 + S_3S_1$이다(여기서 +는 논리 OR). K=1이면 하위 4비트에 0110을 더하고, K=0이면 그대로 둔다. 두 번째 보정 가산기의 carry-out을 K_2라 하면 최종 십진 carry-out은 $K_1 \vee K_2$이다. 예를 들어 9+9는 첫 번째 가산기에서 1_0010, 보정 후 하위 자릿수는 1000이 되며, 첫 번째 가산기의 carry를 유지하여 최종 BCD 결과 0001_1000을 얻는다.